Project report: Making Transformers Solve Compositional Tasks

Younes Boukacem Yassine Ait Said

1 Overview of the Reference Paper

In our chosen paper, namely *"Making Transformers Solve Compositional Tasks"* by Ontañón et al. (2022), the authors propose to study the impact of architectural choices on the ability of Transformers (Vaswani et al., 2023) to generalize in out-of-distribution (OOD) scenarios. Specifically, they propose an experimental setup in which they train small Transformer models to perform specific, toy synthetic tasks—such as duplicating or reversing a string—with fine grained control over the distribution of the training and test data, mainly, the size of the input string. In their setup, the authors make sure that the test data is OOD relatively to the training data, i.e. that the characteristics of the input in the test examples are unseen during training (for example, longer strings). Generalizing to such tasks, which the authors call "compositional generalization", is a persisting problem for developing robust deep learning models. By exploring a design space that includes positional encoding, architecture constraints, and task-specific mechanisms, the authors show that certain configurations substantially improve generalization in such OOD contexts, revealing the high sensitivity to inductive biases induced by model design and training choices in Transformers even if they all rely on the same attention mechanism.

2 Our contributions

To extend the author's work, we identify 3 key limitations: **(1)** They only study encoder-decoder Transformers, which are actively superseded by decoder-only Transformers in LLMs **(2)** The OOD generalization tests were conducted by extending the inputs size of the synthetic tasks, but did not consider extending the inputs number **(3)** Learnable absolute positional encoding was not considered in the study, as they only use fixed sinusoidal absolute encodings and compare it to *learnable* relative positional encodings. In this context, we propose an extension study that **(1)** Focuses on decoder-only Transformers **(2)** Proposes an OOD test protocol which explores augmenting the length of the input strings *and also* the number of input segments, evaluating both in isolation and in combination **(3)** Compares between *learnable* positional encoding mechanisms for both absolute and relative schemes, in addition to two training objectives, namely standard next-token prediction v.s. auto-regressive input masking, which was not considered in the paper.

3 Designing The Toy Synthetic Tasks

As in the original paper, we begin by formalizing our toy synthetic tasks. We design four string manipulation tasks:

1. **Duplication:** corresponds to copying the input strings one by one.
2. **Reverse:** corresponds to copying the input strings on by one but in reverse order (right to left)
3. **Concatenation:** corresponds to concatenating the input strings to form a single string.
4. **Interleaving:** corresponds to a column wise concatenation of equally sized input strings; i.e. the first letters of all the input strings are concatenated, then the second letters are also concatenated and joined with the first ones etc.

These tasks are better visualized in Figure 1. One common point they share is that they all possess an input region which may contain one to several input strings, and an output region containing the output corresponding to the specific task. Both regions are delimited with special markers visible in Figure 1. The model is expected to generate auto-regressively the full output region correctly, given the input region in its context window.

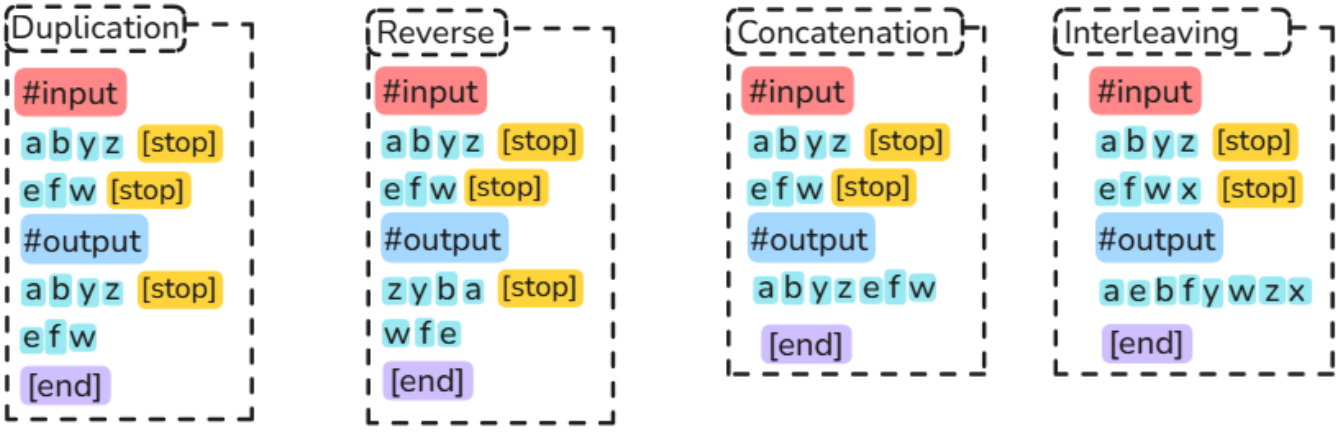

Figure 1: Examples of our proposed synthetic compositional tasks: duplicating, reversing, concatenation, and interleaving.

4 In-Distribution and Out-of-Distribution Protocol

The in-distribution (InD) data configuration restricts the inputs length to [1 to 5] characters and the number of inputs in the [2 to 5] range. The OOD configuration exceeds these ranges to be in [6 to 10] for both character length and number of inputs. We test 5 different combinations of these extended OOD ranges, namely:

- **More-Inputs:** OOD number of inputs. Keeps the InD inputs length.
- **One-Longer-Input:** OOD input length for a single input. Keeps the InD number of inputs.
- **All-Longer-Inputs** OOD input length for all inputs. Keeps the InD number of inputs.
- **(More-Inputs + One-Longer-Input)** and **(More-Inputs + All-Longer-Input)**: straightforward combinations as described by their name.

This protocol are illustrated in Figure 2.

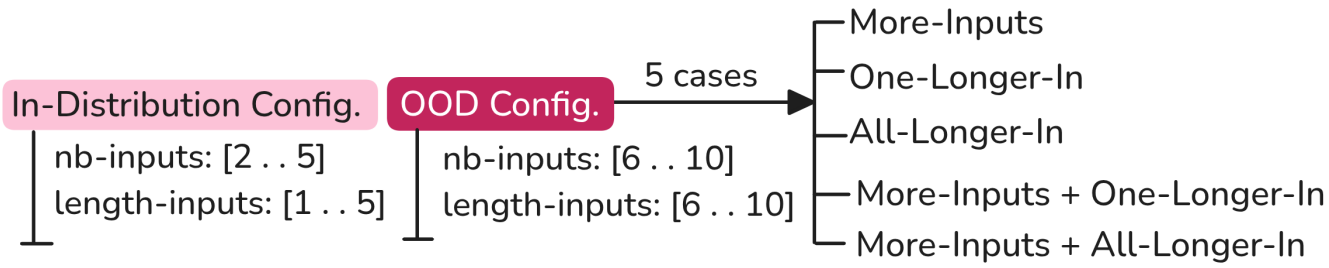

Figure 2: In-distribution and out-of-distribution configurations. ID uses short inputs and few segments; OOD regimes increase the number of inputs, the input length, or both, including combined shifts that stress compositional extrapolation.

5 Model

We target the GPT architecture, and we first identify a base model configuration—illustrated in Figure 3—capable of achieving almost perfect InD-accuracy $> 99\%$ over 10K test samples. The motivation for almost perfect InD-accuracy is that assessing OOD capacity shouldn't make sense for an underperforming model in the InD regime. This point was not highlighted in the original paper.

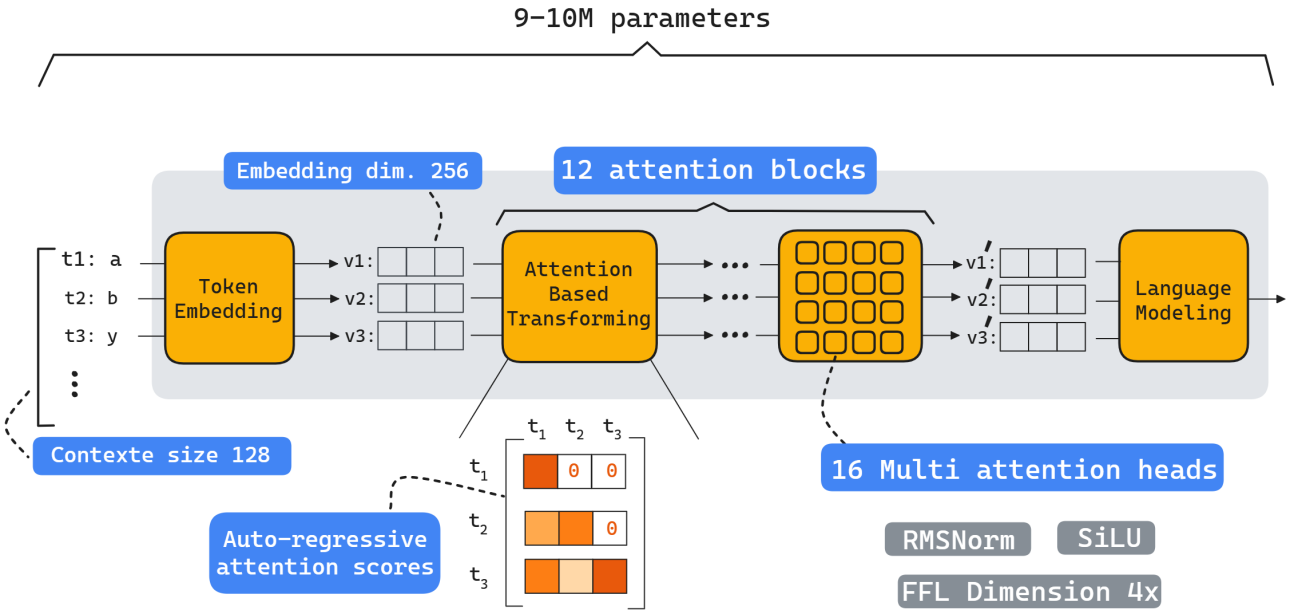

Figure 3: Model overview. Compact decoder-only Transformer (≈9 to 10M parameters) with context window 128, RMSNorm, SiLU activations, and multi-head self-attention of 12 blocks and 16 heads.

6 Compared Architectural Design Choices

6.1 Positional Representations: APE, RPE, RPB

We compare three ways of injecting position information into the decoder-only Transformer. Absolute positional embeddings (APE) add a learnable vector depending on each token's absolute index in the sequence. Relative positional embeddings (RPE) inject learnable vectors that depend on the relative offset between query and key positions inside the attention computation, making attention patterns naturally expressible in terms of distances rather than absolute indices. Finally, relative positional bias (RPB) replaces vectors by learnable scalars that directly bias attention logits as a function of relative offsets. These 3 mechanisms are illustrated in Figure 4.

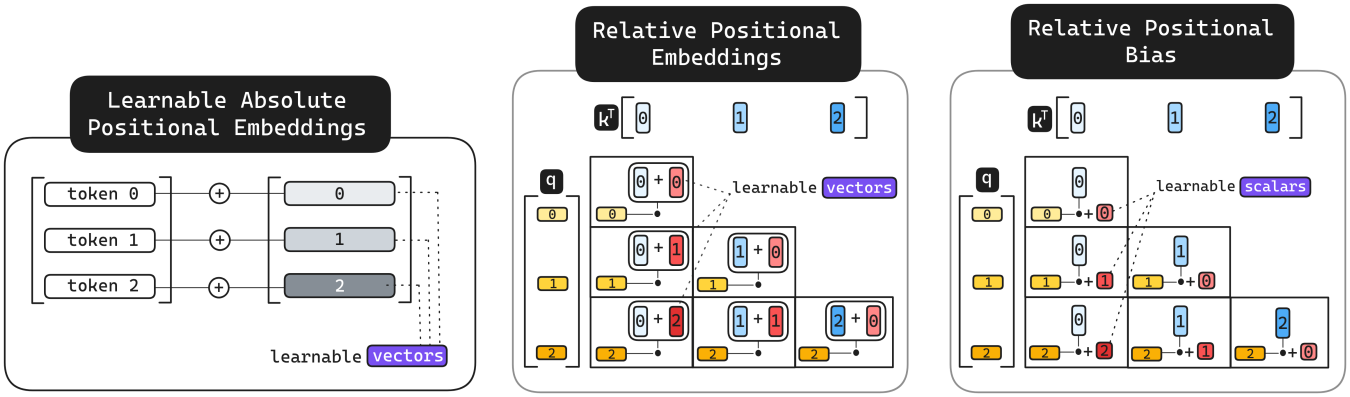

Figure 4: Computational diagram or APE, RPE and RPB.

6.2 Training Objectives: Next-Token Predicition and Input Masking

Encoder-decoder (ED) Transformers have a natural inductive bias that separates input, laying in the encoder, from the output which is generated by the decoder. Conversely, decoder-only models lack this capacity as they only perform next-token prediction (NTP) without additional supervision which may hinder generalization. To explore this effect, we propose to implement an auto-regressive input masking strategy when training our GPT models to mimic ED's bias by ignoring the tokens present in the input region from the model's cross-entropy loss. This is illustrated in Figure 5.

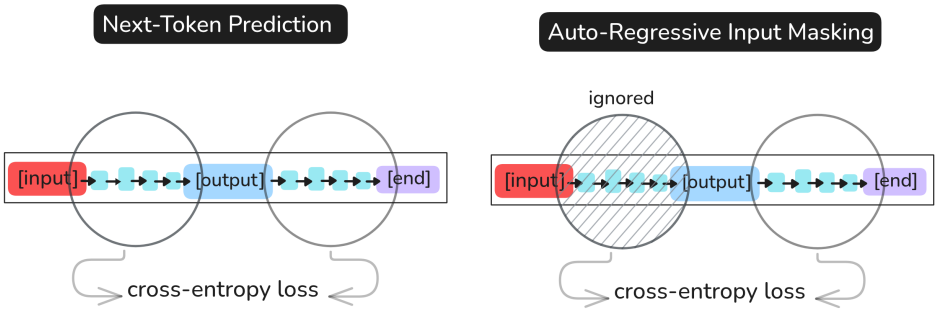

Figure 5: Comparison between NTP and Input-Masking.

7 Results and Insights

7.1 Evaluation setup

For each configuration of positional encoding, training strategy, and test set, we evaluate the accuracy of our GPT model and report the results in Figure 6. Specifically, for each synthetic task, we train our 3 GPT models (with APE, RPE, and RPB) via next-token prediction for achieving the required near-perfect accuracy on the In-Distribution test set. Then we further train each of the three obtained models using Input-Masking, thus obtaining a total of six models for each task all trained on our In-Distribution configuration data described in Figure 2. We then test each of these six models on the OOD benchmarks that were also described in Figure 2, each benchmark containing 10K test samples.

7.2 Results analysis

The integrality of the results are presented in the tables shown in Figure 9 in Appendix A. We focus here on an analysis of those results by highlighting relevant parts of the tables that support our conclusions.

a. Variance in OOD degradation despite high InD accuracy: Focusing first on the models trained with next-token prediction **only**, we find that despite almost-perfect accuracy for all tasks and models, there is strong variance in accuracy degradation when moving to OOD benchmarks. For example, as shown in Figure 6, the APE model delivers accuracies ranging from 0.00% to 92.69% for the OOD tests on concatenation, while this same model plateaus under 6.42% for the reverse task. Such behavior is also observed with RPE and RPB, showing that high in-distribution scores do not always reflect that the model captured the necessary concepts that compose the task it is required to perform.

Concatenation

	In-Distribution			More-Inputs			One-Longer-In			All-Longer-In			More-Inputs + One-Longer-In			More-Inputs + All-Longer-In		
	APE	RPE	RPB	APE	RPE	RPB	APE	RPE	RPB	APE	RPE	RPB	APE	RPE	RPB	APE	RPE	RPB
without Input-Masking	99.99 %	100 %	99.99 %	60.78 %	89.91 %	50.92 %	92.69 %	99.92 %	99.40 %	40.94 %	81.48 %	74.30 %	33.93 %	73.32 %	38.64 %	0.00 %	11.34 %	0.45 %

Reversing

	In-Distribution			More-Inputs			One-Longer-In			All-Longer-In			More-Inputs + One-Longer-In			More-Inputs + All-Longer-In		
	APE	RPE	RPB	APE	RPE	RPB	APE	RPE	RPB	APE	RPE	RPB	APE	RPE	RPB	APE	RPE	RPB
without Input-Masking	99.85 %	100 %	99.99 %	0.17 %	8.04 %	16.46 %	6.42 %	17.71 %	0.01 %	0.23 %	0.69 %	0.00 %	0.00 %	0.33 %	0.00 %	0.00 %	0.00 %	0.00 %

Figure 6: Example of variance in OOD degradation despite high InD accuracy

b. Rarely outperforming APE: Learnable absolute positional embeddings could outperform relative encodings only occasionally, with the only cases where this happened being illustrated by Figure 7. This further corroborates the insight presented by the original paper that absolute positional encodings may be less suited for OOD generalization compared to relative encodings, and our work further extends this insight to learnable absolute encodings on decoder-only models.

Interleaving

	In-Distribution			More-Inputs			One-Longer-In			All-Longer-In			More-Inputs + One-Longer-In			More-Inputs + All-Longer-In		
	APE	RPE	RPB	APE	RPE	RPB	APE	RPE	RPB	APE	RPE	RPB	APE	RPE	RPB	APE	RPE	RPB
without Input-Masking	99.97 %	100 %	100 %	**12.56 %**	0.00 %	0.44 %				0.00 %	0.00 %	0.00 %				0.04 %	0.00 %	0.00 %

Duplicating

	In-Distribution			More-Inputs			One-Longer-In			All-Longer-In			More-Inputs + One-Longer-In			More-Inputs + All-Longer-In		
	APE	RPE	RPB	APE	RPE	RPB	APE	RPE	RPB	APE	RPE	RPB	APE	RPE	RPB	APE	RPE	RPB
without Input-Masking	99.97 %	100 %	99.99 %	14.36 %	6.34 %	16.84 %	13.60 %	15.37 %	0.00 %	**0.94 %**	0.00 %	0.00 %	**1.86 %**	0.06 %	0.00 %	0.00 %	0.00 %	0.00 %

Figure 7: The rare cases of APE outperforming both RPE and RPB

c. Significant improvement through input-masking: Moving on to the effect of additional training via input-masking, we find that it does not degrade InD accuracy while dramatically improving OOD performance but only when base OOD accuracy (i.e. with next token prediction only) isn't too low, as it may otherwise backfire and decrease the OOD score. This is illustrated in Figure 8 where we observe major improvements for concatenation (red boxes) in which models perform well, contrasting with performance decrease in the reversing task (purple boxes) where all models perform badly.

Concatenation

	In-Distribution			More-Inputs			One-Longer-In			All-Longer-In			More-Inputs + One-Longer-In			More-Inputs + All-Longer-In		
	APE	RPE	RPB	APE	RPE	RPB	APE	RPE	RPB	APE	RPE	RPB	APE	RPE	RPB	APE	RPE	RPB
without Input-Masking	99.99 %	100 %	99.99 %	60.78 %	89.91 %	50.92 %	92.69 %	99.92 %	99.40 %	40.94 %	81.48 %	74.30 %	33.93 %	73.32 %	38.64 %	0.00 %	11.34 %	0.45 %
with Input-Masking	100 %	100 %	100 %	75.12 %	98.71 %	97.90 %	99.48 %	99.98 %	99.99 %	49.42 %	98.54 %	85.37 %	47.00 %	97.43 %	92.81 %	0.00 %	86.82 %	2.78 %

Reversing

	In-Distribution			More-Inputs			One-Longer-In			All-Longer-In			More-Inputs + One-Longer-In			More-Inputs + All-Longer-In		
	APE	RPE	RPB	APE	RPE	RPB	APE	RPE	RPB	APE	RPE	RPB	APE	RPE	RPB	APE	RPE	RPB
without Input-Masking	99.85 %	100 %	99.99 %	0.17 %	8.04 %	16.46 %	6.42 %	17.71 %	0.01 %	0.23 %	0.69 %	0.00 %	0.00 %	0.33 %	0.00 %	0.00 %	0.00 %	0.00 %
with Input-Masking	100 %	100 %	100 %	9.92 %	0.00 %	18.40 %	9.20 %	8.52 %	0.00 %	0.03 %	0.23 %	0.00 %	0.70 %	0.00 %	0.00 %	0.00 %	0.00 %	0.00 %

Figure 8: Effect of additional training with input-maksing (second row) compared to bare next-token prediction training (first row)

8 Conclusion, Limitations and Future Work

Our extension work shows that, under a controlled ID/OOD protocol, all tested models may solve in-distribution data nearly perfectly, yet OOD will still reveal strong dependence on inductive biases such as the nature of positional encoding or the training objective, which may have effects that differ depending on the targeted task and data distribution shift.

Regarding the limitations of our study and possible future works, we mention the limited number of positional encoding techniques that we tested, which encourages us to further explore the many more remaining such as RoPE and ALiBi. Also, it should be noted that our proposed input masking technique is more supervised than mere next-token prediction. We should also investigate architectures that can discover high level concepts such as input/ouput separation in a more self-supervised way, such as hierarchical reasoning models and concept models.

References

Ontañón, S., Ainslie, J., Cvicek, V., & Fisher, Z. (2022). Making transformers solve compositional tasks. https://arxiv.org/abs/2108.04378

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2023). Attention is all you need. https://arxiv.org/abs/1706.03762

A Integrality of the evaluation results

Figure 9 presents the integrality of the results obtained for evaluation. Please note that since the interleaving task requires all the input strings to be of the same size, we do not create OOD benchmarks of type *One-Longer-Input* as this would be undefined.

Concatenation

	In-Distribution			More-Inputs			One-Longer-In			All-Longer-In			More-Inputs + One-Longer-In			More-Inputs + All-Longer-In		
	APE	RPE	RPB	APE	RPE	RPB	APE	RPE	RPB	APE	RPE	RPB	APE	RPE	RPB	APE	RPE	RPB
NTP	99.99 %	100 %	99.99 %	60.78 %	89.91 %	50.92 %	92.69 %	99.92 %	99.40 %	40.94 %	81.48 %	74.30 %	33.93 %	73.32 %	38.64 %	0.00 %	11.34 %	0.45 %
IM	100 %	100 %	100 %	75.12 %	98.71 %	97.90 %	99.48 %	99.98 %	99.99 %	49.42 %	98.54 %	85.37 %	47.00 %	97.43 %	92.81 %	0.00 %	86.82 %	2.78 %

Interleaving

	In-Distribution			More-Inputs			One-Longer-In			All-Longer-In			More-Inputs + One-Longer-In			More-Inputs + All-Longer-In		
	APE	RPE	RPB	APE	RPE	RPB	APE	RPE	RPB	APE	RPE	RPB	APE	RPE	RPB	APE	RPE	RPB
NTP	99.97 %	100 %	100 %	12.56 %	0.00 %	0.44 %				0.00 %	0.00 %	0.00 %				0.04 %	0.00 %	0.00 %
IM	100 %	100 %	100 %	0.00 %	0.00 %	0.00 %				0.00 %	0.00 %	0.00 %				0.00 %	0.00 %	0.00 %

Reversing

	In-Distribution			More-Inputs			One-Longer-In			All-Longer-In			More-Inputs + One-Longer-In			More-Inputs + All-Longer-In		
	APE	RPE	RPB	APE	RPE	RPB	APE	RPE	RPB	APE	RPE	RPB	APE	RPE	RPB	APE	RPE	RPB
NTP	99.85 %	100 %	99.99 %	0.17 %	8.04 %	16.46 %	6.42 %	17.71 %	0.01 %	0.23 %	0.69 %	0.00 %	0.00 %	0.33 %	0.00 %	0.00 %	0.00 %	0.00 %
IM	100 %	100 %	100 %	9.92 %	0.00 %	18.40 %	9.20 %	8.52 %	0.00 %	0.03 %	0.23 %	0.00 %	0.70 %	0.00 %	0.00 %	0.00 %	0.00 %	0.00 %

Duplicating

	In-Distribution			More-Inputs			One-Longer-In			All-Longer-In			More-Inputs + One-Longer-In			More-Inputs + All-Longer-In		
	APE	RPE	RPB	APE	RPE	RPB	APE	RPE	RPB	APE	RPE	RPB	APE	RPE	RPB	APE	RPE	RPB
NTP	99.97 %	100 %	99.99 %	14.36 %	6.34 %	16.84 %	13.60 %	15.37 %	0.00 %	0.94 %	0.00 %	0.00 %	1.86 %	0.06 %	0.00 %	0.00 %	0.00 %	0.00 %
IM	100 %	100 %	100 %	22.95 %	25.03 %	21.18 %	26.76 %	82.19 %	0.00 %	1.62 %	30.78 %	0.00 %	5.89 %	8.62 %	0.00 %	0.00 %	0.00 %	0.00 %

Figure 9: Integrality of the evaluation results.